

UNIVERSIDADE AUTÓNOMA DE LISBOA

Mestrado em Engenharia Informática e de Telecomunicações
Programação Avançada de Dispositivos Móveis

RELATÓRIO DE PROJETO

PADM-UAL

Jogo de dados “D20 Pig” para Android

Desenvolvido em Kotlin com Jetpack Compose

Mário Pinheiro

N.º 30015420

Repositório: github.com/mrapinheiro/dice-game

Ano letivo 2025/2026

Índice

1. Introdução	2
1.1. Contexto e enquadramento.....	2
1.2. Objetivos	2
2. O jogo: D20 Pig	2
2.1. Regras.....	2
2.2. Níveis de dificuldade	3
3. Funcionalidades.....	3
4. Tecnologias utilizadas	4
5. Arquitetura da aplicação.....	4
5.1. Visão geral.....	4
5.2. Fluxo de dados unidirecional (MVVM).....	4
5.3. Máquina de estados (GamePhase)	5
5.4. Estrutura do projeto.....	5
6. Detalhes de implementação	5
6.1. Renderização 3D do dado (icosaedro)	5
6.2. Animação de lançamento	6
6.3. Inteligência do oponente (CPU)	6
6.4. Sensor: lançar agitando (shake-to-roll).....	6
6.5. Áudio: efeitos e música	7
6.6. Feedback háptico e ciclo de vida	7
7. Interface do utilizador	8
8. Compilação e execução	8
9. Conclusão.....	9
10. Referências	9

1. Introdução

1.1. Contexto e enquadramento

Este relatório documenta o projeto desenvolvido no âmbito da unidade curricular de Programação Avançada de Dispositivos Móveis, do Mestrado em Engenharia Informática e de Telecomunicações. O objetivo do trabalho foi conceber e implementar uma aplicação Android nativa, completa e jogável, que demonstrasse o domínio das tecnologias modernas de desenvolvimento móvel — nomeadamente a linguagem Kotlin e o toolkit declarativo Jetpack Compose.

A aplicação, designada **PADM-UAL**, é um jogo de dados de casino baseado na variante *D20 Pig*, em que o jogador humano disputa partidas “um contra um” com um oponente controlado pelo computador. O projeto partiu do *codelab* oficial *Dice Roller* da Google como ponto de arranque, tendo sido depois profundamente reescrito e expandido com lógica de jogo original, renderização tridimensional do dado, áudio gerado proceduralmente e interação por sensores.

1.2. Objetivos

Os principais objetivos definidos para o projeto foram os seguintes:

- Implementar uma aplicação Android nativa com uma arquitetura limpa, organizada e testável, seguindo o padrão recomendado pela Google.
- Construir toda a interface de forma *declarativa* com Jetpack Compose e Material 3, sem recurso a XML de layout.
- Explorar capacidades avançadas do dispositivo, em particular o **acelerómetro** (lançar o dado agitando o telemóvel), o **feedback háptico** e a **síntese de áudio** em tempo real.
- Desenvolver um oponente automático com diferentes níveis de dificuldade, recorrendo a *corrotinas* de Kotlin para gerir o seu turno de forma assíncrona e cancelável.
- Garantir o respeito pelo ciclo de vida da aplicação, evitando fugas de memória e libertando recursos de áudio e de sensores nos momentos adequados.

2. O jogo: D20 Pig

O D20 Pig é uma variante do clássico jogo de dados Pig, jogada com um dado de vinte faces (d20) em vez do habitual dado de seis. Trata-se de um jogo de “pressionar a sorte” (push-your-luck): em cada turno o jogador acumula pontos, mas arrisca perdê-los todos se a sorte lhe correr mal. Vence o primeiro a atingir 100 pontos.

2.1. Regras

- Os jogadores alternam turnos; o jogador humano joga primeiro.
- Em cada lançamento, o valor do d20 (de 2 a 20) é somado ao **total do turno**. O jogador pode então optar por lançar de novo ou parar.
- Se sair **1**, todo o total do turno é perdido e a vez passa ao adversário (*bust*).

- Ao **parar (Hold)**, o total do turno é “depositado” na pontuação permanente e a vez passa ao adversário.
- O primeiro jogador a alcançar 100 pontos ganha a partida.

O ponto de tensão do jogo reside precisamente nesta decisão: quanto mais alto o total do turno, maior a recompensa de continuar, mas também maior o prejuízo caso saia um 1. Saber quando parar é a essência da estratégia.

2.2. Níveis de dificuldade

Como a única decisão real do oponente é quando deixar de lançar e depositar os pontos, cada nível de dificuldade corresponde a um limiar de paragem (hold threshold): o computador continua a lançar até o total do turno atingir esse limiar. Num d20, cada lançamento só “estoura” em 5% das vezes e adiciona, em média, cerca de 11 pontos, pelo que limiares mais altos produzem um adversário mais ganancioso e mais rápido a pontuar.

Nível	Limiar de paragem	Comportamento
Fácil	10	Deposita cedo — lento, cauteloso e fácil de ultrapassar.
Médio	20	Estratégia plana quase ótima do Pig; joga sempre da mesma forma, independentemente do resultado.
Difícil	$20 + (\text{vantagem} \div 5)$, limitado a 12–30	Adapta-se ao marcador: sobe o limiar para pressionar quando está à frente e fecha o jogo mais depressa; baixa-o para acumular pontos seguros quando está atrás.

No nível Difícil, $\text{vantagem} = \text{pontuação do CPU} - \text{pontuação do jogador}$. Assim, uma vantagem de 25 pontos eleva o limiar para 25, enquanto estar 50 pontos atrás o reduz ao mínimo de 12. Esta lógica está concentrada no método `CpuStrategy.shouldHold`.

3. Funcionalidades

A aplicação reúne um conjunto alargado de funcionalidades que vão muito além do codelab original:

- **Dado d20 tridimensional** — um verdadeiro icosaedro desenhado sobre um Canvas com projeção em perspetiva, em que todas as 20 faces são visíveis e a face sorteada “aterra” virada para a câmara.
- **Animação de lançamento** — o dado salta, gira rapidamente e assenta com uma desaceleração suave; a face sorteada brilha a dourado depois de estabilizar.
- **Lançar agitando** (shake-to-roll) — através do acelerómetro, agitar o telemóvel equivale a tocar no botão *Roll*.
- **Feedback háptico** — vibração no momento em que o dado assenta e quando há vitória.
- **Efeitos sonoros e música** — efeitos gerados como tons PCM à frequência de amostragem nativa do dispositivo, mais música de fundo em ciclo para os ecrãs de menu e de jogo.
- **Oponente CPU** — com turno automático e três níveis de dificuldade.

- **Estética de mesa de casino** — fundo de pano verde, esquema dourado e disposição edge-to-edge com margens ajustadas às barras de sistema.

4. Tecnologias utilizadas

O quadro seguinte resume a pilha tecnológica do projeto.

Componente	Tecnologia / versão
Linguagem	Kotlin 2.2.10
Interface	Jetpack Compose (Material 3), Compose BOM 2024.12.01
Navegação	Navigation Compose 2.8.5
Ciclo de vida	Lifecycle / ViewModel 2.8.7, Activity Compose 1.9.3
Sistema de compilação	Android Gradle Plugin 9.2.1, Gradle 9.4.1
SDK mínimo	24 (Android 7.0)
SDK alvo / compilação	35 (Android 15)
JDK	17
Arquitetura	Single-Activity + ViewModel + Navigation Compose
Áudio	AudioTrack / SoundPool / MediaPlayer

5. Arquitetura da aplicação

5.1. Visão geral

A aplicação é construída sobre uma **única Activity** (MainActivity) que aloja uma árvore de interface em Compose. Os três ecrãs — menu, regras e jogo — estão ligados através do **Navigation Compose**; a dificuldade escolhida é passada como argumento de navegação tipado (`game/{difficulty}`) e entregue ao `GameViewModel` através do seu `SavedStateHandle`.

5.2. Fluxo de dados unidirecional (MVVM)

O estado flui numa só direção. O `GameViewModel` detém um `MutableStateFlow<GameUiState>` e expõe-no como um `StateFlow` apenas de leitura; a interface observa esse estado e desenha-o, devolvendo intenções ao `ViewModel` (`roll()`, `hold()`, `resetGame()`). Toda a lógica do jogo — totais de turno, a regra de perda ao sair 1 e a verificação de vitória aos 100 pontos — reside no `ViewModel`, o que mantém os *composables* puramente declarativos.

Os **efeitos colaterais** estão desacoplados da lógica: o `ViewModel` expõe `callbacks` anuláveis (`onDiceRollSound`, `onHapticLand`, ...) que a interface liga ao `SoundManager` e ao vibrador do sistema. Desta forma, o som e a vibração nunca tocam no estado do jogo, e o `ViewModel` fica livre de dependências de Android, permanecendo testável de forma isolada.

5.3. Máquina de estados (GamePhase)

Uma enumeração GamePhase funciona como máquina de estados que determina, a cada instante, quais as ações válidas:

```

WAITING_TO_ROLL → ROLLING → ROLLED → (HOLD)
                        ↓
                    TURN_LOST (saiu 1)

qualquer estado → GAME_OVER (um jogador atinge 100)

```

O turno do CPU corre como uma **corrotina cancelável** (cpuTurnLoop) que lança o dado, aplica CpuStrategy.shouldHold e deposita ou estoura. Essa corrotina é cancelada automaticamente ao reiniciar a partida e em onCleared(), evitando que mute o estado de um novo jogo e prevenindo fugas de memória.

5.4. Estrutura do projeto

O código está organizado por responsabilidade, refletindo a separação entre modelo, interface e serviços de dispositivo:

```

app/src/main/java/com/example/diceroller/
├── audio/           SoundManager – SFX em PCM + música (MediaPlayer)
├── model/           GameUiState, CpuStrategy
├── navigation/      NavGraph
├── sensor/          ShakeDetector – gesto de agitar (acelerómetro)
├── ui/
│   ├── menu/       MenuScreen, seletor de dificuldade
│   ├── game/       GameScreen, GameViewModel, D20Canvas,
│   │               DiceAnimations, GameOverOverlay
│   ├── rules/      RulesScreen
│   └── theme/       Color, Type, Theme
└── MainActivity.kt  configuração do NavHost

```

6. Detalhes de implementação

6.1. Renderização 3D do dado (icosaedro)

O dado não é uma imagem: é um icosaedro calculado e desenhado em tempo real no D20Canvas. A geometria é definida por 12 vértices construídos a partir da *razão de ouro* $\phi = (1 + \sqrt{5}) / 2$ e por 20 faces triangulares, cada uma associada ao seu número (1 a 20).

No carregamento da classe são pré-calculadas duas estruturas. Primeiro, as *normais* de cada face. Em segundo lugar, e por cada número de face, a rotação-alvo (rotX, rotY) que melhor orienta essa face para a câmara — obtida por pesquisa exaustiva sobre uma grelha de 72×72 ângulos (passos de 5°). Como esta tabela é construída uma única vez, a operação de lançar torna-se uma simples consulta instantânea.

A cada fotograma, o renderizador executa, por esta ordem:

- aplica as matrizes de rotação em X, Y e Z aos 12 vértices e projeta-os para coordenadas 2D;

- descarta as faces voltadas para trás (*back-face culling*), desenhando apenas aquelas cuja normal aponta para o observador;
- ordena as faces visíveis por profundidade (*algoritmo do pintor*, por ordenação por inserção), garantindo o desenho correto;
- calcula a luminosidade de cada face a partir da sua normal, escolhendo entre três tonalidades para simular iluminação;
- desenha o número no centroide de cada face — a face sorteada a dourado, com brilho (*BlurMaskFilter*) e contorno; as restantes a branco, com transparência proporcional à luminosidade.

Uma preocupação central foi o **desempenho**: objetos *Paint* e vetores *FloatArray* são reutilizados e pré-alocados, evitando a criação de objetos a cada fotograma e, com isso, a sobrecarga do recolector de lixo durante a animação.

6.2. Animação de lançamento

A animação recorre a objetos *Animatable* do *Compose* para rotação (X, Y e Z), escala, deslocamento vertical e desfoque da sombra, e desenrola-se em três fases:

- **Arremesso** (~300 ms) — o dado eleva-se da mesa, aumenta de escala e a sombra cresce.
- **Volteio** (~800 ms) — rotação rápida nos três eixos (720°, 540° e 360°), com *easing linear*.
- **Aterragem** (~500 ms) — os ângulos acumulados são normalizados para o intervalo [0°, 360°] e animados para a frente até à rotação-alvo, de modo a preservar o sentido do giro e a apresentar a face sorteada virada para a câmara; um ligeiro efeito de “esmagar e esticar” dá peso ao impacto.

Concluída a aterragem, a face sorteada é realçada. A saída de um 1 é assinalada por um breve pulso visual do dado, acompanhado da indicação “*Busted!*” no ecrã.

6.3. Inteligência do oponente (CPU)

Como descrito na secção 2.2, a “inteligência” do CPU resume-se à decisão de quando depositar, modelada como um limiar de paragem. O turno é conduzido pela corrotina *cpuTurnLoop*, que introduz uma pausa de “reflexão” com alguma variação aleatória — para o oponente não parecer robótico — antes de cada lançamento. Após cada resultado, o ciclo consulta *CpuStrategy.shouldHold* e decide continuar ou parar; aceita sempre uma vitória garantida, independentemente do limiar. No nível Difícil, o limiar adapta-se dinamicamente à diferença de pontuação, tornando o adversário mais agressivo em vantagem e mais conservador em desvantagem.

6.4. Sensor: lançar agitando (shake-to-roll)

O *ShakeDetector* é um *SensorEventListener* autónomo que escuta o acelerómetro. Para distinguir uma agitação deliberada do simples andar ou de pequenos toques, aplica um **filtro passa-baixo** (coeficiente 0,8) que estima a gravidade em cada eixo; subtraindo-a, obtém a aceleração linear. Só quando a magnitude desta ultrapassa um limiar ($\approx 13 \text{ m/s}^2$, cerca de 1,3 g) é registado um gesto.

Para robustez, o detetor descarta as primeiras amostras enquanto o filtro converge (*priming*) e impõe um **período de espera** de 600 ms entre agitações, de modo a que um único movimento corresponda

a um único lançamento. As notificações são entregues no fio principal através de um Handler, e o detetor é **sensível ao ciclo de vida**: é iniciado em ON_RESUME e parado em ON_PAUSE. O gesto só desencadeia um lançamento quando é a vez do humano e nenhum lançamento está em curso.

6.5. Áudio: efeitos e música

O SoundManager combina três mecanismos. Dois efeitos amostrados — o rolar e o assentar do dado — são reproduzidos via SoundPool. Os restantes efeitos são **sintetizados proceduralmente** como PCM (vetores ShortArray), à frequência de amostragem nativa do dispositivo — obtida de PROPERTY_OUTPUT_SAMPLE_RATE — para evitar artefactos do reamostrador do sistema.

A síntese assenta em três geradores:

- generateTone — uma senoide com envelope de fade-in/fade-out (ex.: clique de botão, “reflexão” do CPU);
- generateSweep — uma varredura de frequência (ex.: depósito de pontos, de 500 a 1200 Hz; perda de turno, de 800 a 300 Hz);
- generateFanfare — uma sequência de notas (vitória: arpejo de Dó maior, 523/659/784/1046 Hz; derrota: sequência descendente).

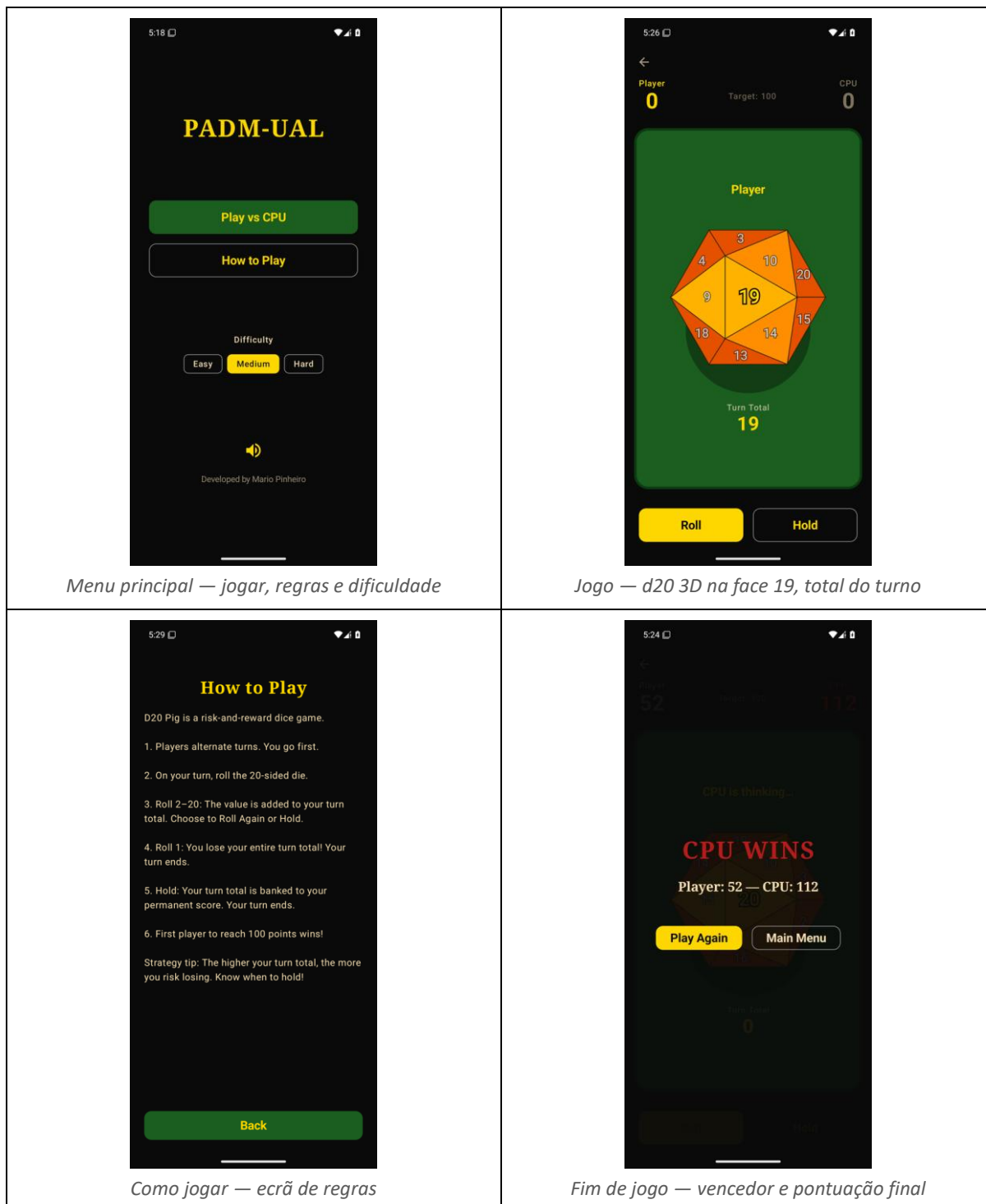
Cada AudioTrack é construído no momento da reprodução, num Executor de fio único (fora do fio principal), e libertado em seguida. A música de fundo em ciclo, para os ecrãs de menu e de jogo, recorre a MediaPlayer.

6.6. Feedback háptico e ciclo de vida

A vibração utiliza o LocalHapticFeedback do Compose (sendo a permissão VIBRATE declarada no manifesto): um impulso longo quando o dado assenta e na vitória, e um impulso curto ao reconhecer a agitação. A gestão de recursos é centralizada na MainActivity, que observa o ciclo de vida e encaminha os eventos ON_PAUSE, ON_RESUME e ON_DESTROY para pausar, retomar ou libertar o áudio. Assim, sons, sensores e vibração são sempre parados e libertados nos momentos corretos.

7. Interface do utilizador

A aplicação organiza-se em três ecrãs principais. As imagens seguintes ilustram o aspeto final da interface, com a estética de mesa de casino adotada em todo o jogo.



8. Compilação e execução

O projeto requer o **Android Studio** (versão Narwhal ou superior) com o **JDK 17**. A compilação e instalação podem ser feitas a partir da linha de comandos com o wrapper do Gradle:

```
./gradlew :app:assembleDebug    # compila o APK de depuração  
./gradlew :app:installDebug     # instala no dispositivo/emulador
```

Em alternativa, basta abrir o projeto no Android Studio e executar diretamente. O código-fonte completo está disponível no repositório indicado na capa.

9. Conclusão

O projeto cumpre os objetivos propostos, resultando numa aplicação Android nativa completa, robusta e jogável. Partindo de um simples codelab, foi possível construir um jogo com lógica original, um oponente automático com estratégia adaptativa e uma camada de apresentação cuidada.

Do ponto de vista da aprendizagem, o trabalho permitiu consolidar competências em diversos domínios avançados do desenvolvimento móvel: a construção de interfaces declarativas com Jetpack Compose; a aplicação do padrão MVVM com fluxo de dados unidirecional; a programação assíncrona com corrotinas; a integração de sensores de hardware com filtragem de sinal; a síntese de áudio em tempo real; e, transversalmente, o respeito rigoroso pelo ciclo de vida da aplicação e pela gestão de recursos.

Os maiores desafios técnicos situaram-se na renderização tridimensional do dado — em particular no cálculo das rotações-alvo e na otimização do desempenho do desenho — e na coordenação das animações com a lógica de jogo e os respetivos efeitos. A separação clara de responsabilidades adotada na arquitetura revelou-se decisiva para manter o código compreensível e extensível à medida que a complexidade aumentava.

10. Referências

Google — *Build your first Android app in Kotlin (codelab “Dice Roller”)*.

developer.android.com/codelabs/basic-android-kotlin-compose-dice-roller

Android Developers — *Jetpack Compose*. developer.android.com/jetpack/compose

Android Developers — *Guide to app architecture (ViewModel, StateFlow)*.

developer.android.com/topic/architecture

M. Pinheiro — *PADM-UAL: dice-game (código-fonte)*. github.com/mrapinheiro/dice-game